

KI-Guidelines für unsere Unit (Hardware-Design)

Diese Guidelines sind für ein Team gedacht, das **keine Software schreibt**, sondern Hardware designt und aufbaut. Die meisten kursierenden KI-Tipps sind auf Coding-Teams zugeschnitten (Prompting fürs Schreiben von Code, Code-Review-Loops etc.) und lassen sich nicht 1:1 übernehmen. Ein paar der zugrundeliegenden Prinzipien — u. a. von Andrej Karpathy, der typische Fehlermuster von Coding-Agents beschrieben hat — sind aber allgemeiner Natur und übertragen sich gut auf unsere Arbeit: Recherche, Berechnungen, Dokumentation, Reviews, Kommunikation.

Ziel dieses Dokuments: eine gemeinsame, unitweite Grundlage, wie wir KI-Tools strukturiert einsetzen — nicht ad hoc und je nach Person unterschiedlich.

0. Grundhaltung

KI ist ein sehr belesener, sehr schneller, aber **unzuverlässiger Junior-Kollege ohne Gedächtnis und ohne Verantwortung**. Sie liefert oft exzellente erste Entwürfe und Recherche-Vorarbeit — aber sie:

- erfindet plausibel klingende Zahlen, Bauteil-Parameter und Norm-Verweise, wenn sie unsicher ist (**Halluzination**), ohne das kenntlich zu machen
- trifft bei unklaren Aufgaben stillschweigend eine von mehreren möglichen Annahmen, statt nachzufragen
- neigt dazu, mehr zu liefern als verlangt (unaufgeforderte Zusatzinhalte, aufgeblähte Formulierungen)
- hat keinen Begriff von "fertig" — sie meldet Ergebnisse als erledigt, ohne sie geprüft zu haben

Die Guidelines unten sind Gegenmaßnahmen für genau diese vier Punkte.

1. Vor dem Prompten: Scope klären statt raten lassen

(entspricht Karpathys "Think Before Coding")

Eine vage Anfrage führt zu einer von der KI geratenen Interpretation — meist die naheliegendste, nicht zwingend die richtige.

Umsetzung:

- Bei nicht-trivialen Aufgaben nicht direkt "mach mal", sondern die KI selbst nach fehlenden Infos fragen lassen, bevor sie loslegt. Prompt-Muster: *"Bevor du anfängst: Welche Annahmen müsstest du treffen, und was fehlt dir, um das sauber zu lösen? Frag nach, statt zu raten."*
 - Kontext explizit mitgeben, den die KI sonst nicht hat: welches Projekt, welcher Bauteil-/Baugruppen-Stand, welche Norm/Kundenanforderung gilt, wer die Zielgruppe des Ergebnisses ist (interner Vermerk vs. Kundendokument vs. Lieferantenanfrage).
 - Bei größeren Aufgaben (z. B. "vergleiche diese fünf Datenblätter und erstelle eine Auswahlmatrix") vorher kurz mit der KI die Kriterien und das Ausgabeformat abstimmen, statt das Ergebnis zu bekommen und dann festzustellen, dass die falschen Spalten drin sind.
-

2. Ergebnisse verifizieren, nicht glauben — besonders bei Zahlen und Normen

(entspricht Karpathys "Goal-Driven Execution" / "fehlende Verifikation")

Das ist für uns der **wichtigste Punkt**, weil falsche Werte hier nicht nur einen Build fehlschlagen lassen, sondern reale physische, sicherheitsrelevante oder Compliance-Konsequenzen haben können (falsche Toleranz, falsche Spannungsfestigkeit, falsch zitierte Norm-Klausel).

Umsetzung:

- Zahlen, Bauteil-Parameter und Norm-Zitate, die eine KI nennt, **immer gegen die Primärquelle prüfen** (Datenblatt, Norm-Text, Messprotokoll) — nie aus dem KI-Gedächtnis übernehmen, auch wenn es überzeugend klingt.
- Bei Recherche-Aufgaben die KI explizit bitten, Quellen/Seitenzahlen/Abschnitte zu nennen, die man dann stichprobenartig gegenprüft — nicht als Beleg, sondern als Ausgangspunkt für die eigene Prüfung.
- Bei sicherheits- oder kundenrelevanten Inhalten (Spezifikationen, Prüfberichte, alles was rausgeht): KI-Entwurf = erster Entwurf, nie finale Freigabe. Reviewschritt durch eine Person bleibt Pflicht.
- Für interne, unkritische Zusammenfassungen (siehe Punkt 5) kann die Prüftiefe geringer sein.

3. Gezielt statt großflächig ändern lassen

(entspricht Karpathys "Surgical Changes")

Wenn eine KI ein bestehendes Dokument (Spezifikation, Stückliste, Bericht) überarbeitet, neigt sie dazu, mehr umzuschreiben als nötig — das macht Änderungen schwer nachvollziehbar und erzeugt neue Fehlerquellen in Abschnitten, die eigentlich gar nicht angefasst werden sollten.

Umsetzung:

- Bei Überarbeitungen explizit sagen, was **nicht** verändert werden soll: *"Ändere nur Abschnitt 3, der Rest bleibt exakt wie er ist."*
- Ergebnis als Diff/Vergleich zum Original anschauen, wenn das Tool das hergibt — nicht das ganze neue Dokument ungeprüft übernehmen.

4. Klare, überprüfbare Ziele statt vager Wünsche

(entspricht Karpathys "Goal-Driven Execution")

Je konkreter das Erfolgskriterium, desto eher liefert die KI etwas Brauchbares — und desto leichter lässt sich hinterher prüfen, ob sie es erreicht hat.

Umsetzung:

- Statt "fass das mal zusammen" lieber: "Fasse die drei Messberichte auf einer Seite zusammen, mit einer Tabelle Ist- vs. Sollwert pro Prüfpunkt."
- Bei größeren, mehrstufigen Aufgaben (z. B. Bauteil-Auswahl über mehrere Lieferanten) Teilziele

definieren, die einzeln prüfbar sind, statt einer einzigen großen Anfrage, deren Ergebnis man am Ende nur noch als Ganzes akzeptieren oder verwerfen kann.

5. Nicht alles muss auf Hochglanz-Niveau geprüft werden

(entspricht Karpathys "Simplicity First", angepasst)

Nicht jede KI-Ausgabe ist gleich kritisch. Es lohnt sich, bewusst zwischen zwei Modi zu unterscheiden, statt für alles denselben Prüfaufwand zu betreiben:

- **"Vibe"-Nutzung** (schnelle interne Notiz, erste Gliederung, Formulierungshilfe, Brainstorming) — geringe Prüftiefe, Zeit sparen.
- **Verbindliche Nutzung** (geht an Kunden/Lieferanten, fließt in eine Spezifikation, betrifft Sicherheit/Compliance) — volle Prüfung nach Punkt 2, keine Abkürzung.

Diese Unterscheidung vorher explizit treffen, nicht erst hinterher merken, dass ungeprüfter Text in einem offiziellen Dokument gelandet ist.

6. Vertraulichkeit und Datenklassifizierung — vor Punkt 1 klären

Bevor überhaupt etwas in ein KI-Tool eingegeben wird:

- Was darf laut Firmenrichtlinie in welches Tool (extern gehostet vs. intern/on-prem)? *[hier unitspezifisch ergänzen, sobald die Regelung von IT/Compliance vorliegt]*
- Vertrauliche Lieferantendaten, Kundenspezifikationen, exportkontrollierte Informationen (ggf. ITAR/EAR-relevant) grundsätzlich **nicht** in öffentliche KI-Tools ohne vorherige Freigabe.
- Im Zweifel: Datenblätter/Normen-Auszüge sind meist unkritisch, firmen- oder kundenspezifische Konstruktionsdaten meist nicht — im Zweifelsfall nachfragen statt annehmen.

7. Konkrete Einsatzfelder für unsere Unit

Damit "wofür können wir KI eigentlich nutzen" nicht länger unklar ist:

Bereich	Beispiele
Recherche	Datenblätter durchsuchen/vergleichen, Normen/Standards nachschlagen und einordnen, Bauteil-Alternativen finden
Analyse & Review	Zweitmeinung zu Designentscheidungen, Plausibilitätscheck von Berechnungen, Messdaten auf Auffälligkeiten prüfen, Spezifikationen auf Lücken/Widersprüche gegenlesen

Dokumentation	Technische Berichte, Prüfprotokolle, Spezifikationsentwürfe, Zusammenfassungen langer Dokumente
Kommunikation	Meeting-Notizen strukturieren, Statusberichte, Entwürfe für Lieferanten-/Kundenkorrespondenz
Auswertung/Automatisierung	Kleine Skripte für Datenauswertung (z. B. Messreihen aus Excel/CSV), CAD-Automatisierung — auch ohne selbst programmieren zu können: KI-generierte Skripte immer von jemandem gegenprüfen lassen, der die Logik nachvollziehen kann, bevor sie auf echten Daten läuft

Kurzfassung zum Aushängen

1. **Scope vorher klären**, nicht raten lassen.
2. **Zahlen und Normen immer an der Primärquelle prüfen** — KI-Gedächtnis ist keine Quelle.
3. **Gezielt ändern lassen**, nicht großflächig umschreiben lassen.
4. **Klare, prüfbare Ziele** statt vager Aufträge.
5. **Prüftiefe bewusst wählen** — intern/unkritisch locker, kunden-/sicherheitsrelevant streng.
6. **Vertraulichkeit zuerst klären**, bevor irgendwas eingegeben wird.

Quelle: [ki-guidelines-hardware-unit.md](#)